Workshop OGC API Processes

Wifi wachtwoord staat op de tafels

Auteur Niels Hoffmann en Pieter Bresters, Geonovum

Datum 10 maart 2026

GEONOVUM

Programma

10:00 – 10:15	Even voorstellen
10:15 – 11:00	Presentatie: introductie OGC API’s
11:00 – 12:00	Praktijk / zelf aan de slag
12:00 – 13:00	Pauze
13:00 – 13:30	Presentatie: inhoudelijke verdieping
13:30 – 16:00	Praktijk / zelf aan de slag

Inleiding op OGC API's

Waarom OGC API's ?

- Makkelijker dan oude generatie services voor web-ontwikkelaars buiten het geodomein
  - Je hoeft geen GIS-specialist te zijn om er mee te werken;
  - Ook goed te gebruiken binnen het geodomein;
  - Openapi specificatie beschrijft wat er mee kan;
  - Werkt heel intuïtief.

- Beter te vinden door zoekmachines

- Verschillende types hebben dezelfde structuur en zijn daardoor ook eenvoudiger te combineren.

Belang Geonovum stimuleren OGC-API-gebruik?

OGC-API's passen binnen de visie van Geonovum:

- Geonovum wil de Nederlandse geo-informatie infrastructuur versterken door het voor een brede groep toegankelijker te maken.

OGC-API spelen een rol in ontwikkelingen die van belang zijn voor Geonovum zoals:

- Europese datastrategie
- IBDS: Federatief Datastelsel
- Common Ground
- Zicht op Nederland
  - Digital Twins
  - Datafundament

Features

Approved Standard

OGC API - Features - Part 1: Core and Part 2: Coordinate Reference Systems by Reference are both publicly available.

Common

Approved Standard

OGC API - Common specifies those building blocks that are shared by most or all OGC API Standards to ensure consistency across the family.

EDR

Approved Standard

Environmental Data Retrieval (EDR) API provides a family of lightweight interfaces to access Environmental Data resources. Each resource addressed by an EDR API maps to a defined query pattern.

Tiles

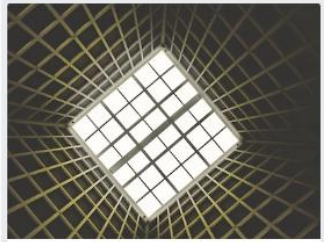
Approved Standard

OGC API - Tiles provides extended functionality to other OGC API Standards to deliver vector tiles, map tiles, and other tiled data.

Processes

Approved Standard

OGC API - Processes allows for processing tools to be called and combined from many sources and applied to data in other OGC API resources though a simple API.



Coverages

OGC API - Coverages allows discovery, visualization and query of complex raster stacks and data cubes.

Records

OGC API - Records updates OGC's Catalog Services for the Web by building on the simple access to content in OGC API - Features.

Styles

The OGC API - Styles defines a Web API that enables map servers, clients as well as visual style editors, to manage and fetch styles.

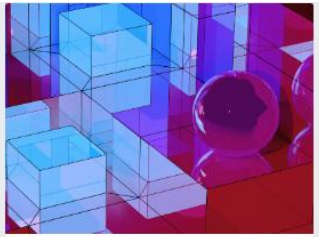
Maps

OGC API - Maps offers a modern approach to the OGC Web Map Service (WMS) standard for provision map and raster content.

WFS > Features
WMS > Maps
WMST > Tiles
CSW > Records
WPS > Processes

Moving Features

OGC API - Moving Features defines an API that provides access to data representing features that move as rigid bodies.



3D GeoVolumes

OGC API - 3D GeoVolumes facilitates efficient discovery of and access to 3D content in multiple formats based on a space-centric perspective.

Connected Systems

OGC API - Connected Systems act as a bridge between static and dynamic data collected by sensors.

DGGS

Enables applications to organise and access data arranged according to a Discrete Global Grid System (DGGS).

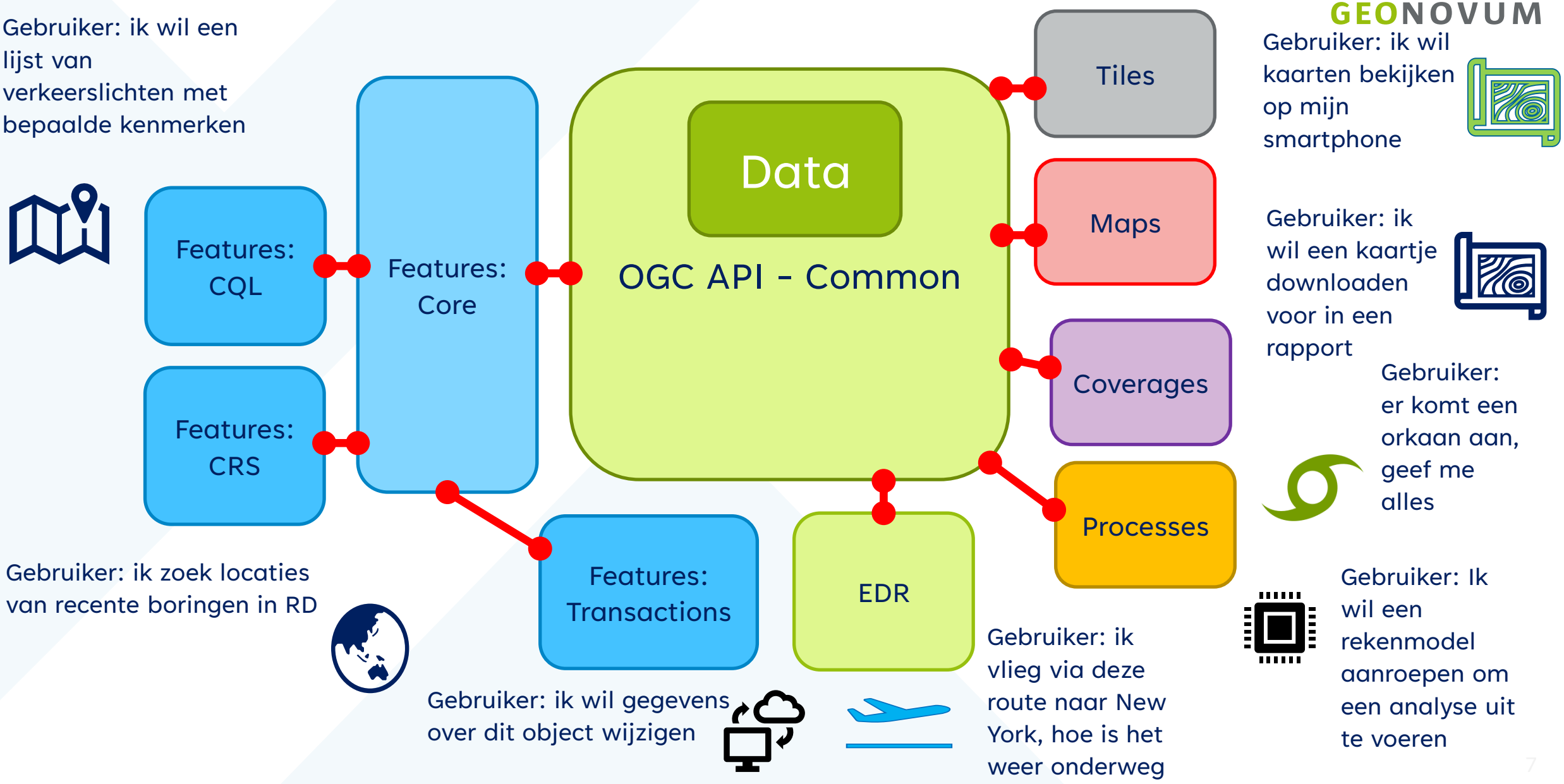
Routes

Enables applications to request routes in a manner independent of the underlying routing data set, routing engine or algorithm.

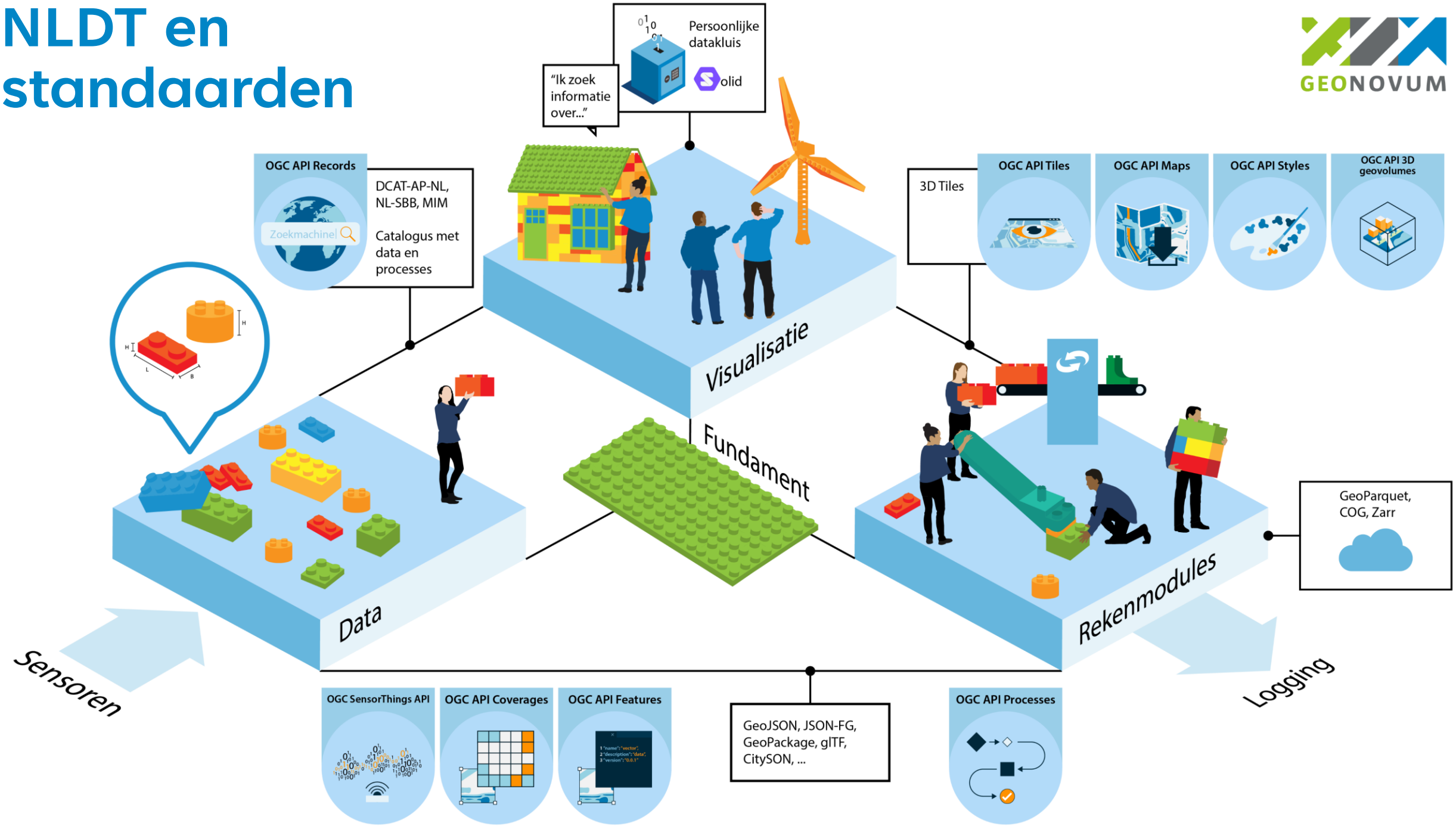
Joins

OGC API - Joins supports the joining of data, from multiple sources, with feature collections or directly with other input files.

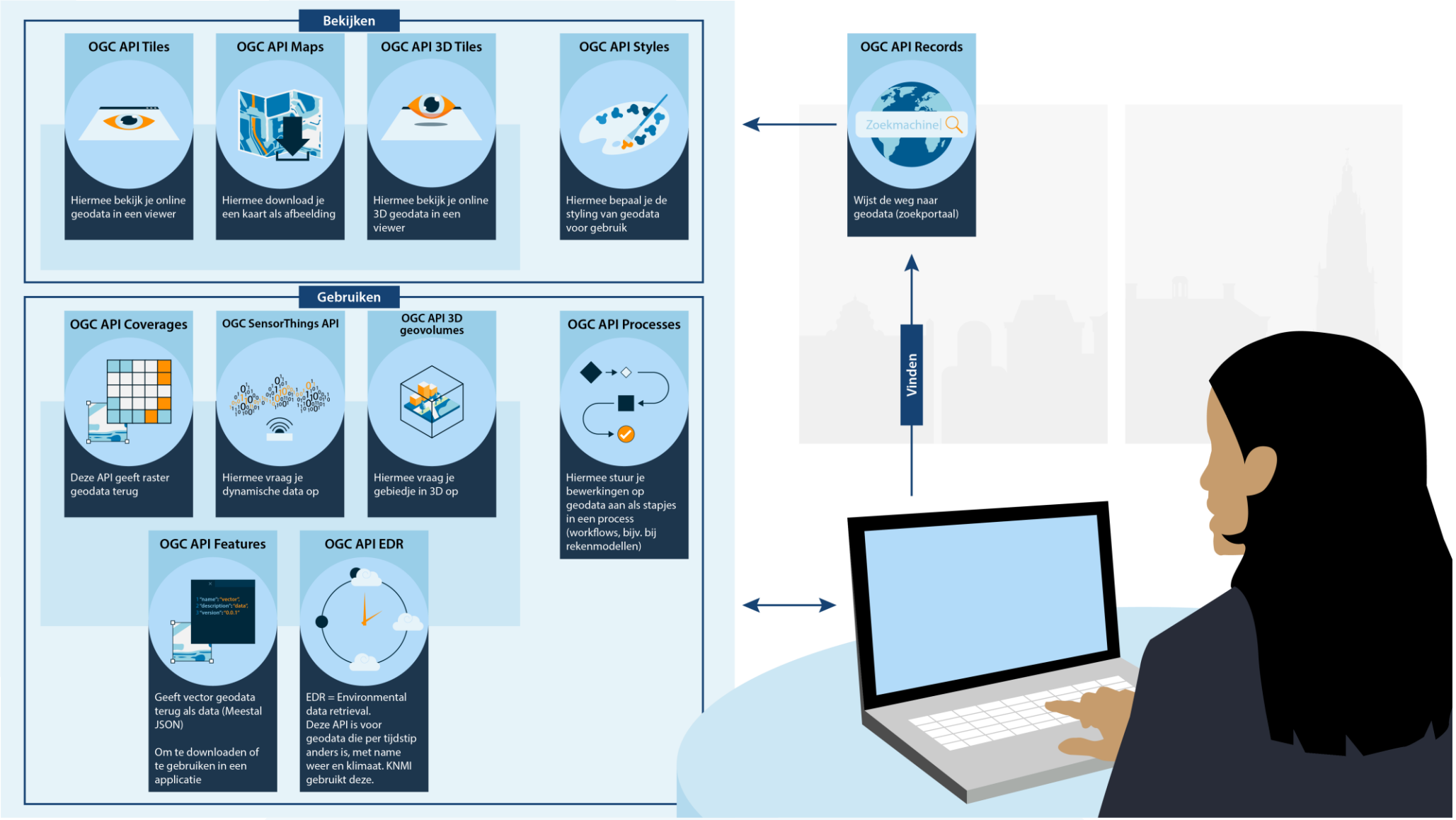
Deployment model voorbeeld – API bouwstenen



NLDT en standaarden



Zoeken, bekijken en gebruiken



OGC API Common – algemene regels

- Consistent gebruik van HTTP(S)
- Consistente URI opbouw
- Consistent gebruik van parameters
- Er is altijd een landing page
- Consistente manier van aanbieden JSON en HTML
- Bij voorkeur API beschrijving in OpenAPI 3.0

verkenning van OGC API-standaarden

Welke Standaarden zijn er voor API's ?

1. Open API Specification (v3.0)
2. OGC
3. Dutch API Design Rules

Open API Specification

```yaml
openapi: 3.0.0
info:
  title: Petstore API
  version: 1.0.0
  description: A sample API for managing pets

paths:
  /pets:
    get:
      summary: List all pets
      responses:
        '200':
          description: A list of pets.
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Pet'
    post:
      summary: Create a new pet
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Pet'
      responses:
        '201':
          description: The created pet.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Pet'
        '400':
          description: Invalid input
```

```yaml
  /pets/{petId}:
    get:
      summary: Get a pet by ID
      parameters:
        - in: path
          name: petId
          schema:
            type: integer
          required: true
          description: ID of the pet to retrieve
      responses:
        '200':
          description: The pet object.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Pet'
        '404':
          description: Pet not found
```

```yaml
components:
  schemas:
    Pet:
      type: object
      required:
        - name
      properties:
        id:
          type: integer
          format: int64
          readOnly: true
          description: The ID of the pet.
        name:
          type: string
          description: The name of the pet.
```

OGC

- Standaard verschilt per vastgesteld type API maar er is een common

- Laatste vastgestelde versie OGC API - Processes - Part 1: Core (1.0): HTML of PDF

- Laatste Draft OGC API - Processes - Part 1: Core (2.0): HTML of PDF
- Laatste Draft OGC API - Processes - Part 2: Deploy, Replace, Undeploy: HTML of PDF
- Laatste Draft OGC API - Processes - Part 3: Workflows and Chaining: HTML of PDF
- Laatste Draft OGC API - Processes - Part 4: Job Management: HTML of PDF

(stand februari 2026) - We kijken in deze workshop alleen naar Part 1: Core 1.0

Dutch API Design Rules

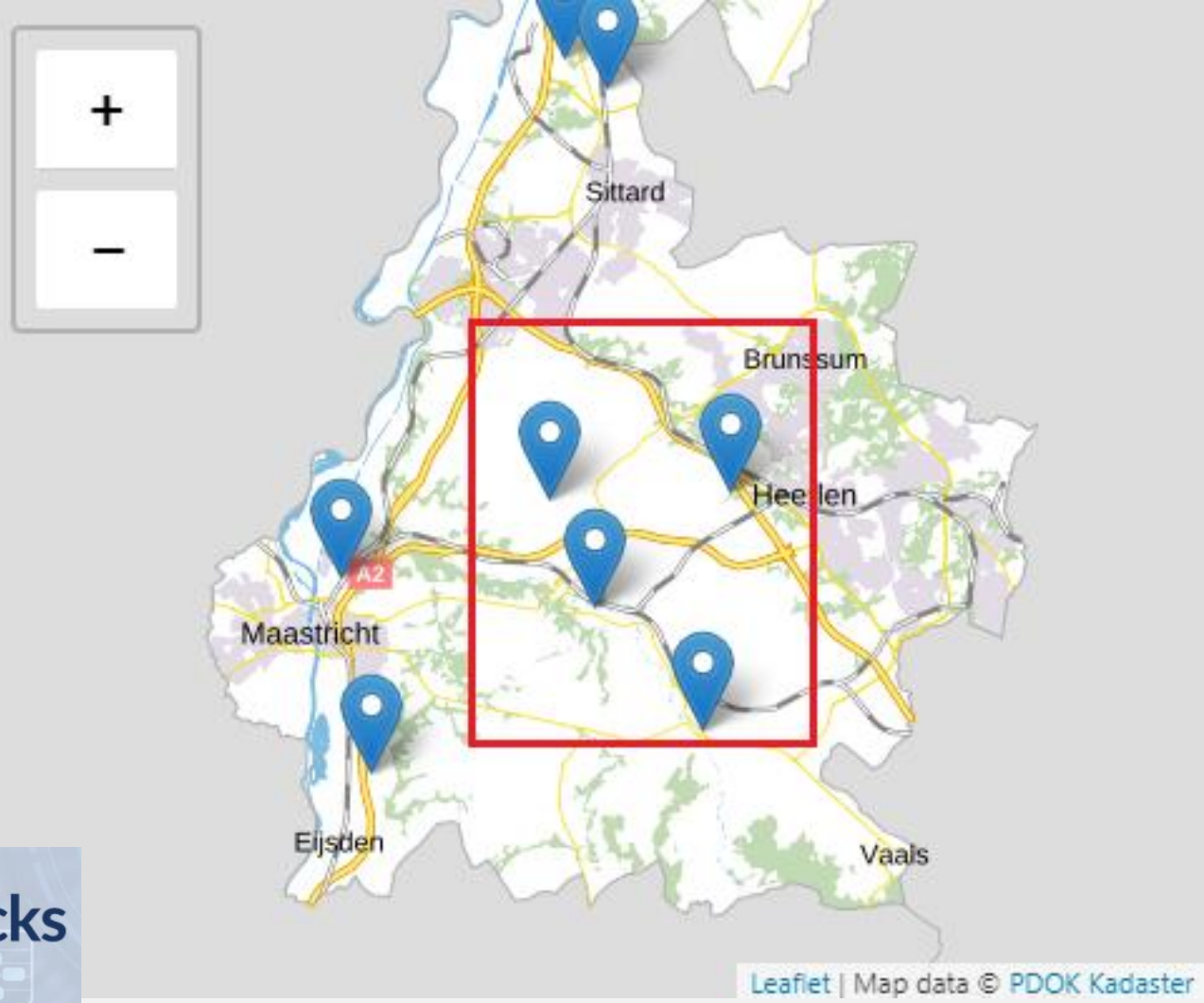
- Komt voort uit het kennisplatform API's
- Wordt door Logius gepubliceerd: API Design Rules
- Geomodule

De geo module

https://docs.geostandaarden.nl/api/API-Strategie-mod-geo/

Regels voor geografische inhoud en functionaliteit in APIs

Toepassing van internationale geo-standaarden zoals GeoJSON, OGC API standaarden, INSPIRE richtlijn



Open Geospatial Consortium

OGC APIs | Building Blocks for Location

Inhoud uit de OGC API Features standaard, deel 1 en 2. [zie https://ogcapi.ogc.org/features/]

```
GET
/api/v1/kastelen/items?bbox=5.4,52.1,5.5,53.2
```

Gebruik OGC-API-Features

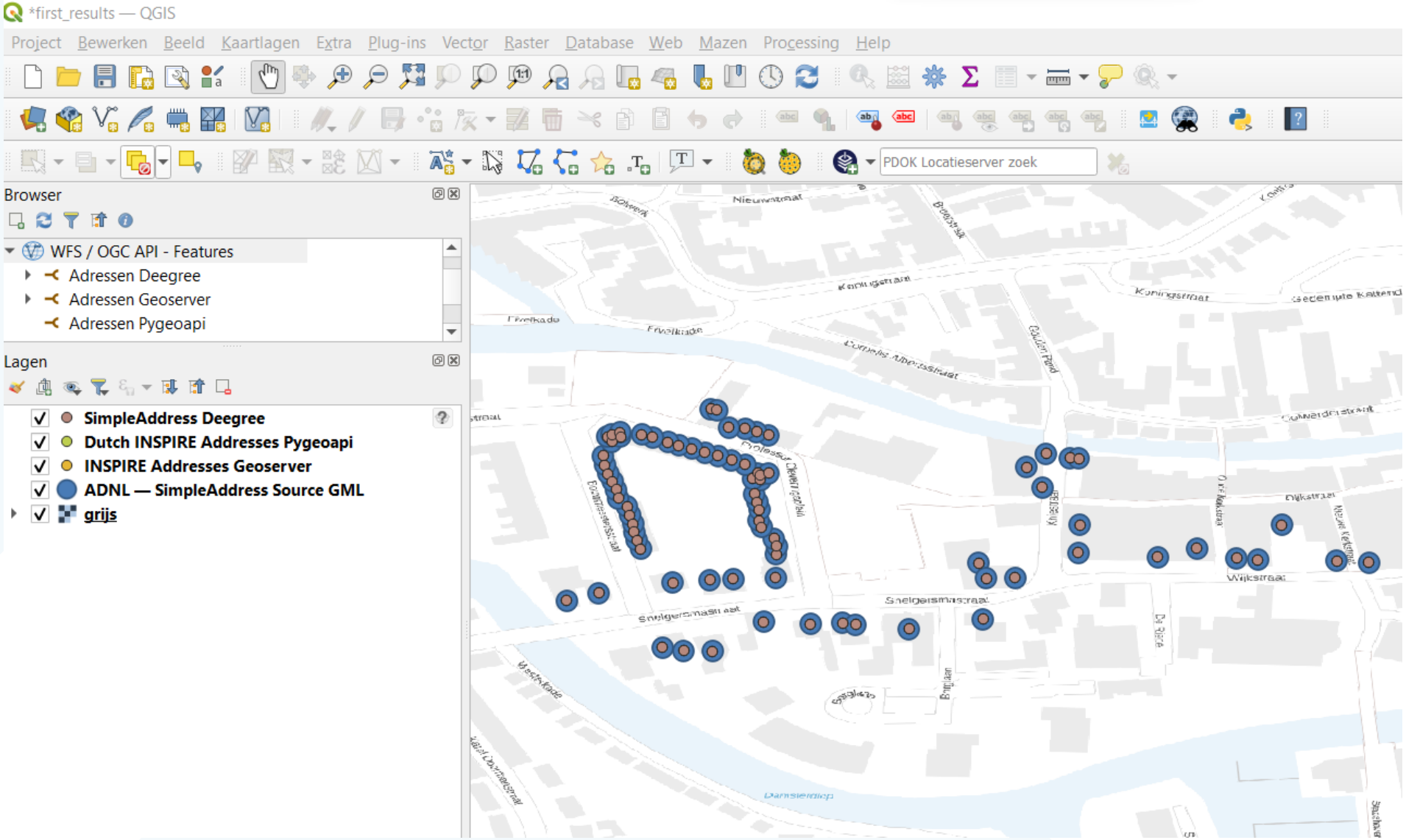
OGC API Features

- Landing page https://api.pdok.nl/lv/bgt/ogc/v1
- OAS https://api.pdok.nl/lv/bgt/ogc/v1/api
- Collections https://api.pdok.nl/lv/bgt/ogc/v1/collections
- Collection-id https://api.pdok.nl/lv/bgt/ogc/v1/collections/bak
- Items https://api.pdok.nl/lv/bgt/ogc/v1/collections/bak/items
- Item-id: https://api.pdok.nl/lv/bgt/ogc/v1/collections/bak/items/5d394ef5-6a5d-5011-a729-29def1c51dd9

Parameters:

- ?f=json of ?f=html
- ?crs vb: ?crs=http://www.opengis.net/def/crs/EPSG/0/28992
- ?bbox vb: ?bbox=160.6,-55.95,-170,-25.89

OGC-API-Features in QGIS



Gebruik OGC-API-Processes

OGC API Processes

Resource	Path	HTTP method	Information delivered
Landing page	/	GET	General information about the service, links to API endpoints
Conformance classes	/conformance	GET	List of conformance classes
Process list	/processes	GET	Process identifiers, links to process descriptions
Process description	/processes/{processID}	GET	Information about a process, e.g. inputs/outputs
Process execution	/processes/{processID}/execution	POST	Create a job (execute a process)
Job status info	/jobs/{jobID}	GET	Status info, links to results or exceptions
Job results	/jobs/{jobID}/results	GET	Job results
Job list	/jobs	POST/GET*	List of job ids and status info, links to results or exceptions

Operations conform Requirements class ‘Core’

* POST is conform Requirements class ‘Core’, GET is conform Requirements class ‘Job List’

OGC API Processes (OAS)

http://localhost:5000/openapi

server Deze instance dient als MVP demo voor OGC API Process logging met OpenTelemetry. information

- GET / Landing page
- GET /collections Collections
- GET /conformance API conformance definition
- GET /openapi This document
- GET /processes Processes

knmi_meetstations knmi meetstations

- GET /collections/knmi_meetstations Get Meetstations metadata
- GET /collections/knmi_meetstations/items Get Meetstations items
- OPTIONS /collections/knmi_meetstations/items Options for Meetstations items
- GET /collections/knmi_meetstations/items/{featureId} Get Meetstations item by id
- OPTIONS /collections/knmi_meetstations/items/{featureId} Options for Meetstations item by id
- GET /collections/knmi_meetstations/queryables Get Meetstations queryables
- GET /collections/knmi_meetstations/schema Get Meetstations schema

OGC API Processes (Python)

GEONOVUM

Python requests

```python
import requests

url = "http://localhost:5000/processes/squared/execution"
data = {"inputs": {"number-or-integer": 42}}
headers = {"Content-Type": "application/json"}

response = requests.post(url, json=data, headers=headers)

print(response.json())
```

✓ 0.0s

```
{'id': 'squared', 'value': 1764}
```

OWSLib (https://owslib.readthedocs.io/en/latest/usage.html#ogc-api-processes-part-1-core-1-0)

```python
from owslib.ogcapi.processes import Processes

client = Processes('http://localhost:5000/')

response = client.execute('squared', inputs={"number-or-integer": 42})

print(response)
```

✓ 0.0s

```
{'outputs': [{'id': 'squared', 'value': 1764}]}
```

Zelf aan de slag met API’s

Zelf experimenteren

Wat gaan we doen?

- Een docker instantie van Pygeoapi opstarten
- Aan deze instantie Processes toevoegen
- De processes opvragen met Jupyter Notebook

- Zie: https://github.com/Geonovum/ogc-api-workshops
  - **05 OGC API Processes**

handson/pygeoapi

Verdiepen OGC-API Processes

Meer functionaliteiten...

- Async execution en Callback
- Server en Processes los van elkaar
  - Implementatie in nodeJS -> glue code tussen processen en api
    - Shell script
    - Node script
    - Docker container

Resources in Core

Inputs:
- Inline
- By reference

Mode:
- Synchronously
- Asynchronously

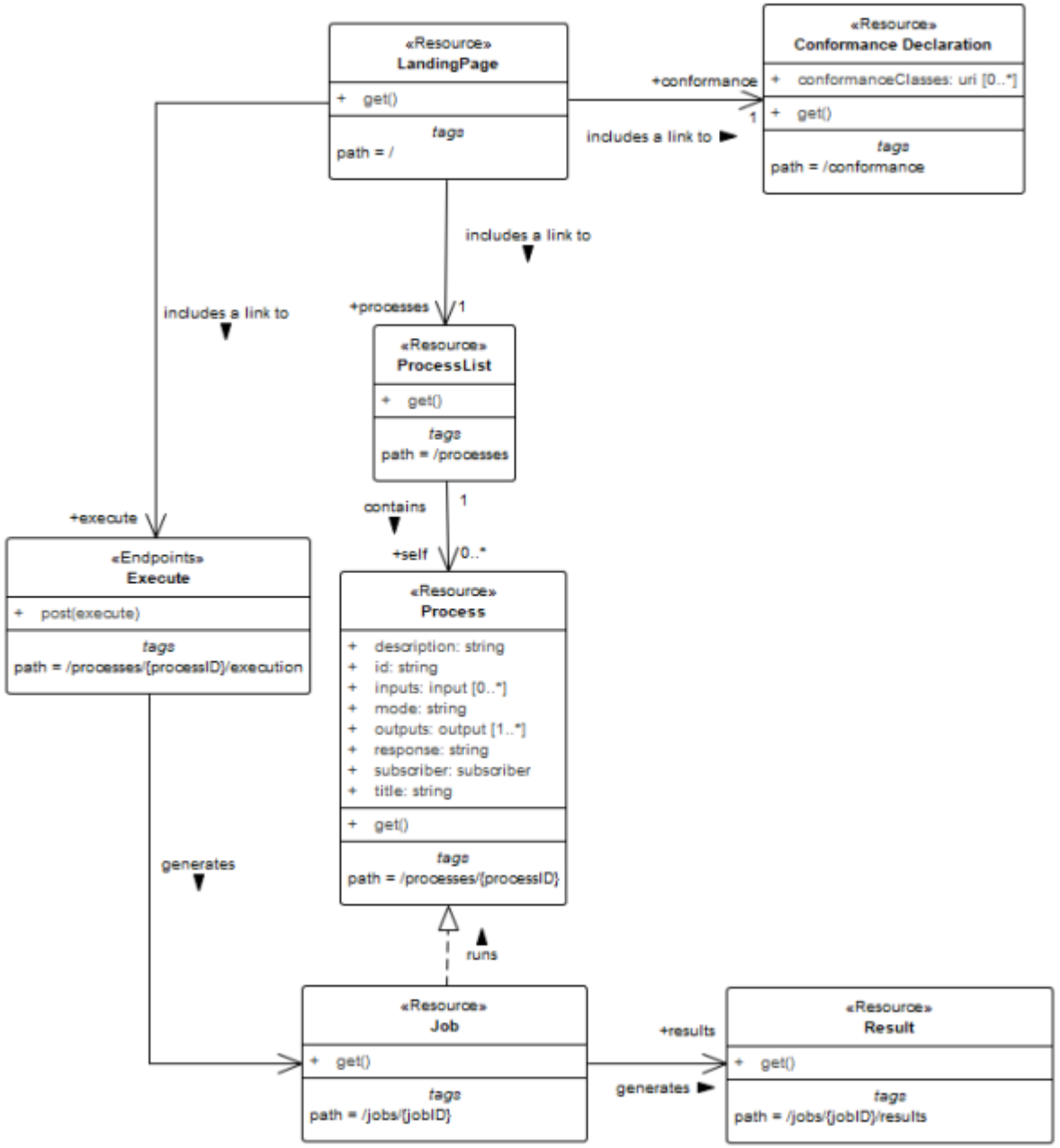
(HTTP Prefer header)

Response:
- Document
- Raw

Outputs:
- Value
- Reference

(transmissionMode)

Async:
- http status 201
- Location header with link to job



Voorbeeld Process Description

OpenAPI 3.0 schema

```
{
  "id": "countFeatures",
  "title": "Count features",
  "description": "Count the total number of features from an API Features endpoint.",
  "version": "0.0.1",
  "jobControlOptions": ["sync-execute"],
  "outputTransmission": ["value", "reference"],
  "inputs": {
    "uri": {
      "title": "Features endpoint",
      "description": "Uri to dataset.",
      "schema": {
        "type": "string",
        "format": "url"
      }
    }
  },
  "outputs": {
    "count": {
      "id": "Result",
      "title": "Feature count",
      "description": "Total number of features provided by OGC API Endpoint.",
      "schema": {
        "type": "number",
        "format": "integer"
      }
    }
  }
}
```

Zelf aan de slag met API’s

deel 2

Zelf experimenteren

Optie1:
Verder met pygeoapi van 1e deel

- Probeer nu zelf een process toe te voegen

- Er staat een basis voor addNumbers die verder uitgewerkt kan worden.

- Pas de pygeoapi.config.yml aan
- Pas de docker-compose.yml aan

Optie2:
NodeJS Demoserver

Experimenteren met verschillende soorten processes

- Shell Script
- NodeJS Script
- Docker container

- Zie: https://github.com/Geonovum/ogc-api-kennissessie
  - OGC-API-Processes/nodejs-demo

- Gebruik een API Client (Postman, Thunderclient, Bruno, ...)
- Bruno: https://www.usebruno.com/downloads
  - Open collection: Bruno\OGC-API

Pygeoapi proces implementeren

- Pygeoapi biedt een ‘raamwerk’ om een proces te implementeren.
  - Om dat in de demo omgeving werkend te krijgen moeten een aantal dingen geregeld worden:
    - Python file die pygeopapi.BaseProcessor implementeert (bijvoorbeeld addNumbers.py)
    - De python file moet gemapt worden in de docker container -> docker compose volume mapping
    - De Class uit de python file moet bekend gemaakt worden in de pygeoapi config.yaml
    - In de python file moet de metadata beschreven worden om het proces in de API/OAS te publiceren

- De andere demo (NodeJS) is eigenlijk alleen een ‘doorgeefluik’ tussen de API en een proces op je laptop.
  - Wat je daar implementeert is ‘flexibeler’, Shell script, code, docker container. Maar je moet dus wel meer zelf ‘regelen’ om aan de OGC API Processes te voldoen.

Meer informatie

- Setup van de demo omgeving: https://dive.pygeoapi.io/setup/
- https://dive.pygeoapi.io/

- Linter: https://geonovum.github.io/ogc-checker/#/ogc-api-processes

- Part2 (Deploy, Replace, Undeploy): https://docs.ogc.org/DRAFTS/20-044.html
- Part3 (Workflows and Chaining): https://docs.ogc.org/DRAFTS/21-009.html
- Part4 (Job Management): https://docs.ogc.org/DRAFTS/24-051.html

Bedankt!

Geonovum

T 033 460 41 00
E info@geonovum.nl
I www.geonovum.nl

bezoekadres
Barchman Wuytierslaan 10
3818 LH Amersfoort

postadres
Postbus 508
3800 AM Amersfoort